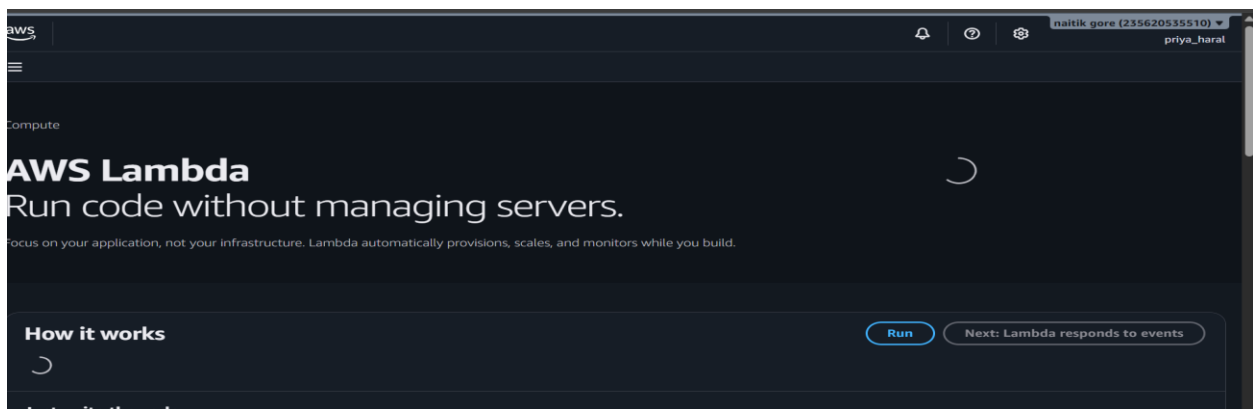


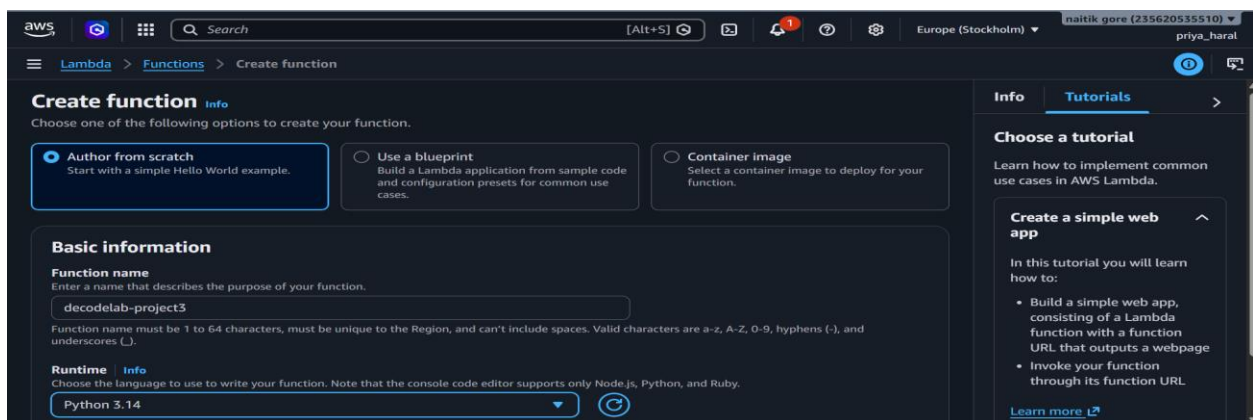
The Serverless Logic

Project Objective

The objective of this project is to build a simple serverless Cost Calculator using AWS Lambda and Python. The project accepts two numbers as input, calculates their sum, returns the result in JSON format, and uses Amazon CloudWatch to monitor and verify the function execution.

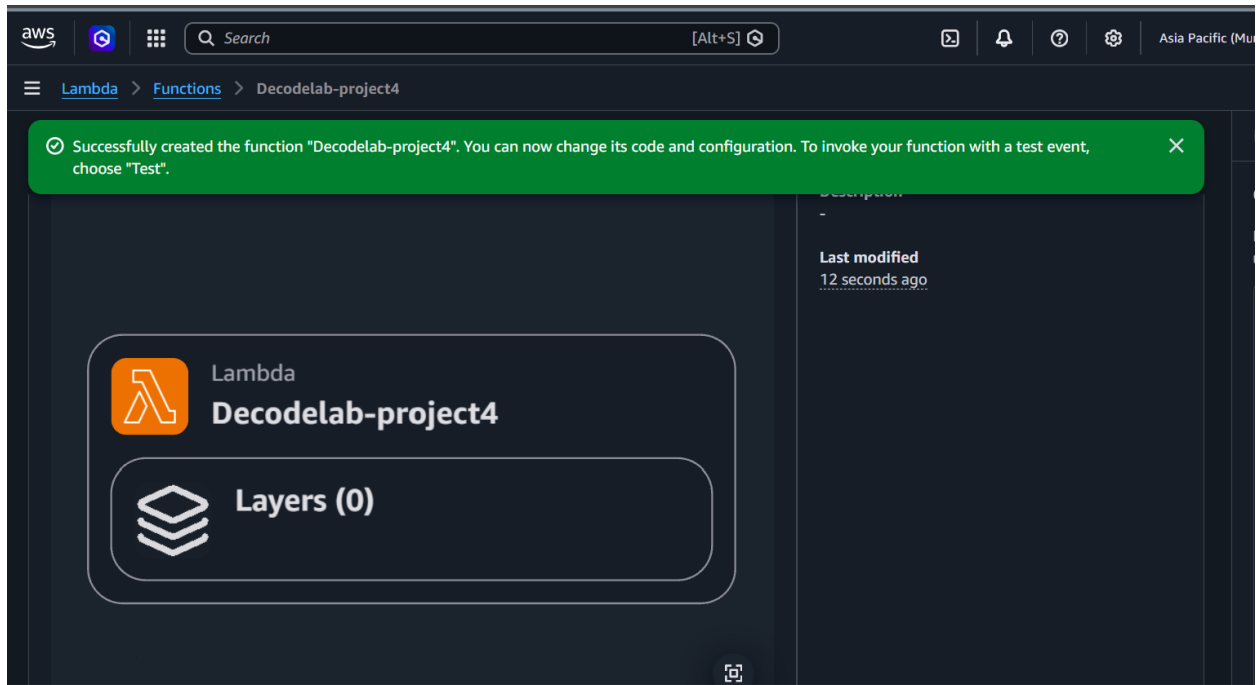


AWS Lambda was selected because it allows code to run without managing servers. It is suitable for event-driven applications and short-running workloads.



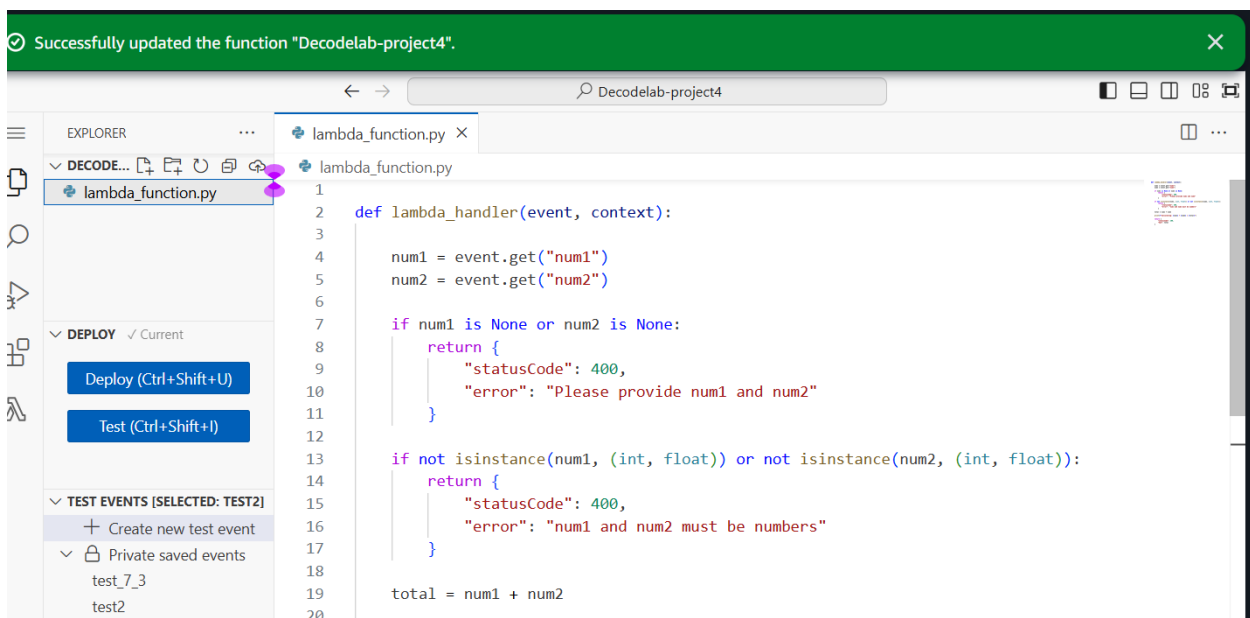
Function Name: **Decodelab-Project4**

A new Lambda function named **Decodelab-Project4** was created from scratch to implement the calculator logic. An IAM execution role was configured to provide the Lambda function with the required AWS permissions. Basic permissions were used for CloudWatch logging.



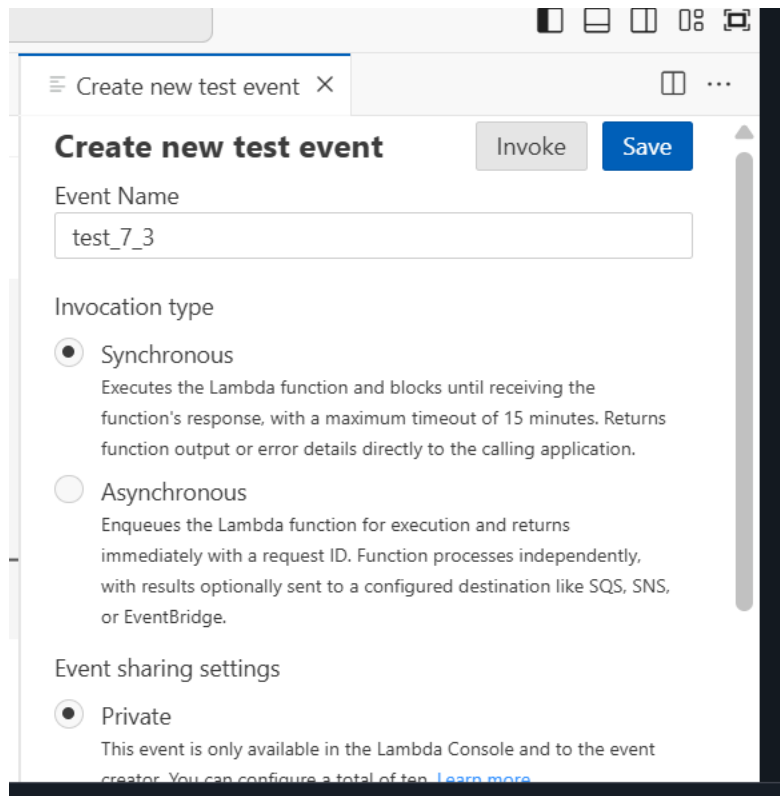
The Python Lambda handler receives two numbers from the event, validates the input, calculates their sum, and returns the result as a JSON response.

The Lambda function was deployed to save and activate the latest version of the code so that it could be executed during testing.



Create Test Event :

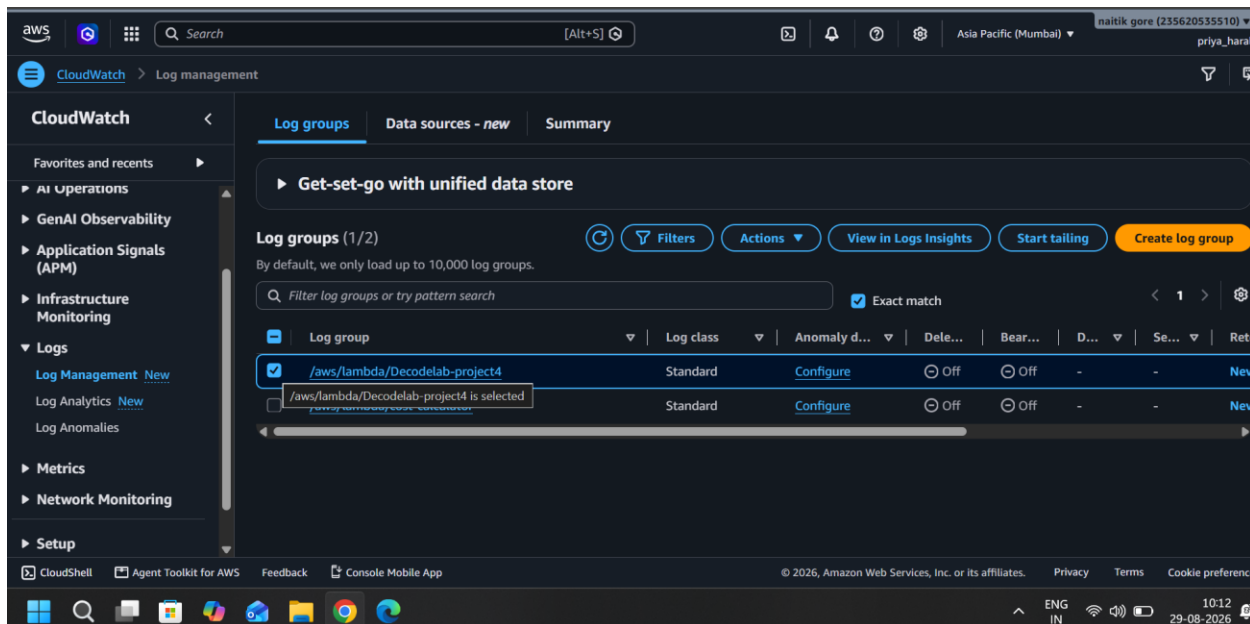
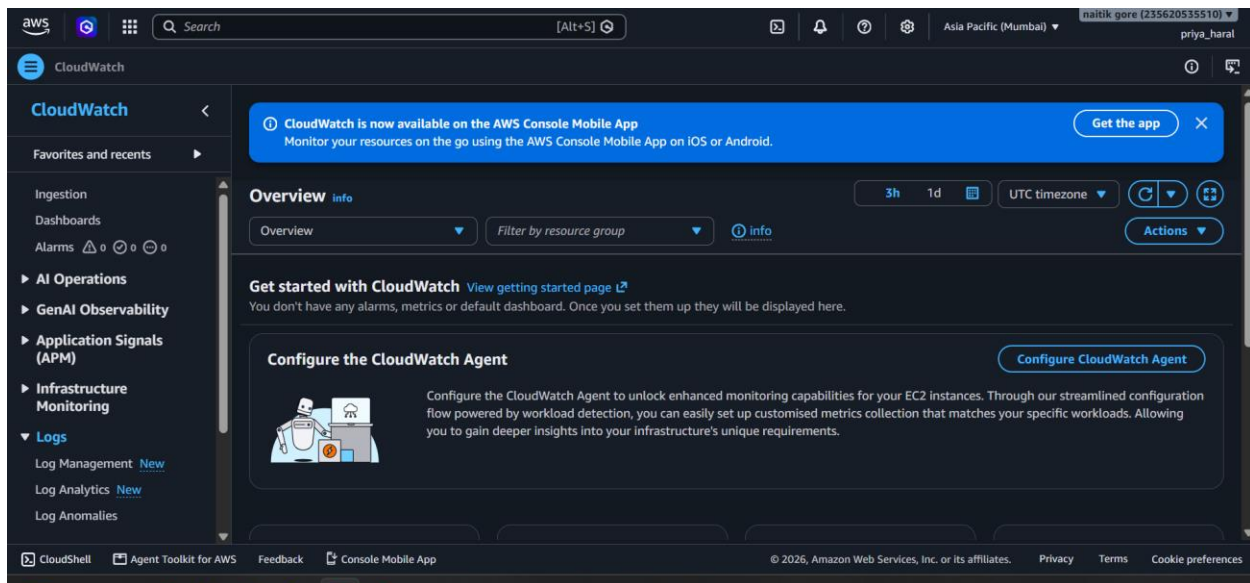
```
{  
  
  "num1": 7,  
  
  "num2": 3  
  
}
```



The screenshot shows the 'Create new test event' window in the AWS Lambda console. The window has a title bar with a hamburger menu, the text 'Create new test event', a close button (X), and window control icons. Below the title bar, there is a section titled 'Create new test event' with two buttons: 'Invoke' and 'Save'. The 'Event Name' field contains the text 'test_7_3'. Under the 'Invocation type' section, the 'Synchronous' radio button is selected, with a description: 'Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.' The 'Asynchronous' radio button is unselected, with a description: 'Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.' Under the 'Event sharing settings' section, the 'Private' radio button is selected, with a description: 'This event is only available in the Lambda Console and to the event creator. You can configure a total of ten. [Learn more](#)'.

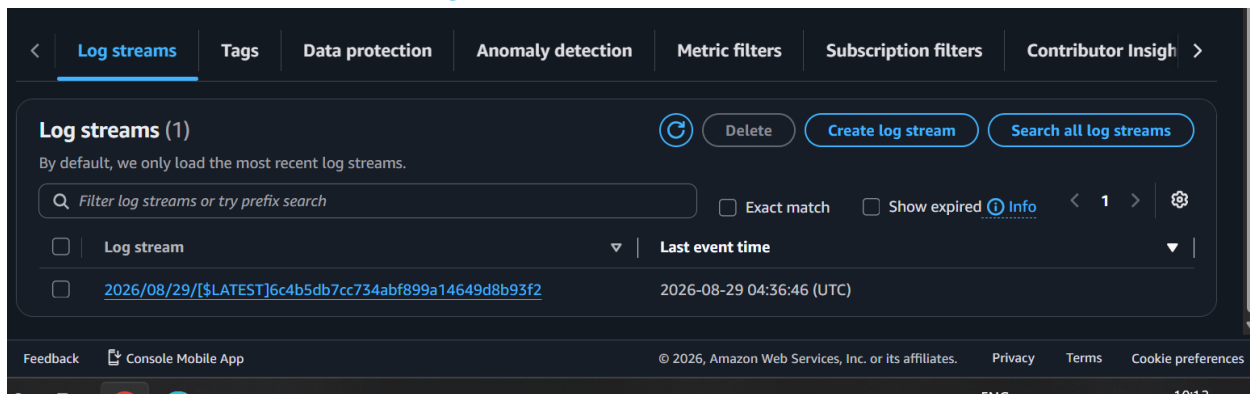
A mock JSON test event was created to provide input values to the Lambda function without requiring an external application or API. The Lambda function was invoked using the test event. AWS executed the Python code using the provided input values.

The CloudWatch logs provide execution evidence, including the calculation output, request information, execution duration, and billed duration.

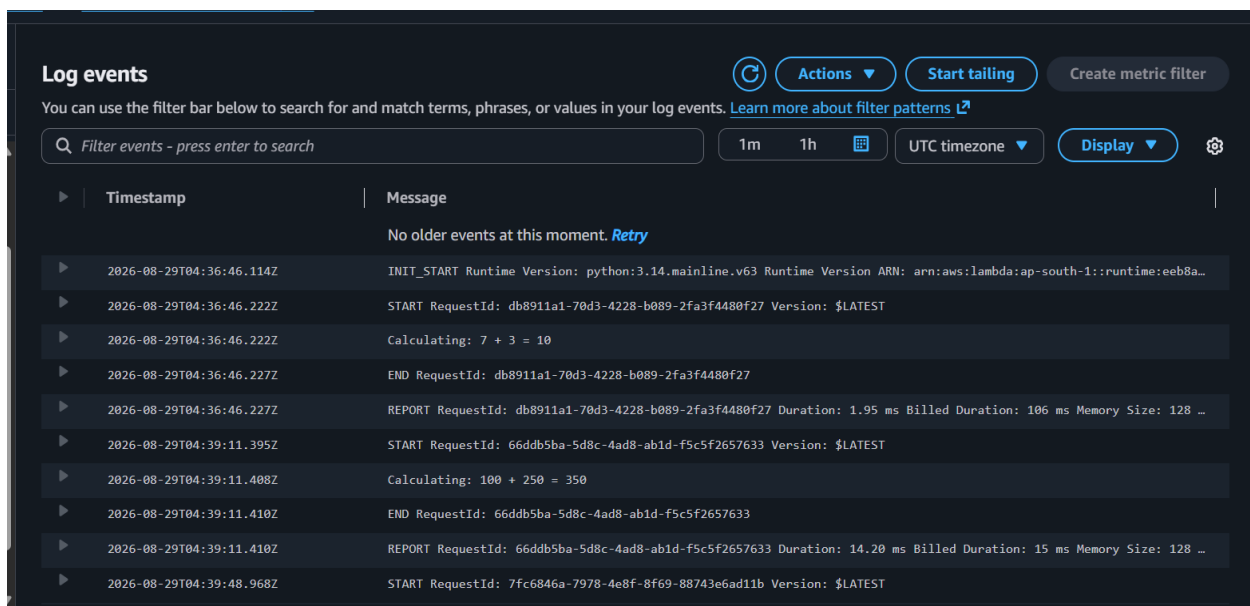


The Lambda log group contains execution records generated during function invocations. The Lambda log group contains execution records generated during function invocations.

/aws/lambda/DecodeLab-project4



CloudWatch Log Stream for the AWS Lambda function. It records the function execution details, including the request ID, calculation output, execution duration, and billed duration. These logs verify that the Lambda function executed successfully



The CloudWatch logs provide execution evidence, including the calculation output, request information, execution duration, and billed duration.

The project successfully demonstrates a serverless calculator that receives input, processes the calculation, returns a JSON response, and records execution information in CloudWatch.

This project successfully demonstrates the implementation of a serverless Cost Calculator using AWS Lambda and Python. The function accepts two numbers,

validates the input, calculates their sum, returns a JSON response, and provides execution evidence through Amazon CloudWatch.